并行算法

主讲人：段贵多
Email:duanguiduo@163.com

第三章 并行程序开发

1. 并行编程环境
2. MPI
3. **矩阵向量乘积和MPI**
4. 矩阵向量乘法并行化

并行编程环境

主要是从编程方式和通信方式分类。

（1）以编程方式分类

- 隐式并行环境；
- 显式并行环境；

（2）以通信方式分类

- 共享存储并行环境；
- 消息传递并行环境；

并行编程环境

主要是从编程方式和通信方式分类。

（1）以编程方式分类

- 隐式并行环境
  - 用串行语言编程并使用某种标记指明并行性和控制流调度，编译器或运行时后环境自动将其转化为并行代码。
  - 编程能力往往受限
- 显式并行环境
  - 编写代码显示地指明程序的控制流的创建、执行、调度和退出。
  - 更难使用，但更自由

并行编程环境

主要是从编程方式和通信方式分类。

（2）以通信方式分类

- 共享存储并行环境
  - 支持多线程并行、使用统一虚拟地址空间通信、线程间使用函数显式同步。
  - 适合多核处理器
- 消息传递并行环境
  - 支持进程并行、使用独立的地址空间、支持异步数据传输、显式同步、显式负载分配、显式通信。
  - 更适合分布式多机环境

并行编程环境

- 支持隐式共享存储器并行环境：OpenMP、OpenACC、SSE/AVX和NEON内置函数.
- 支持显式共享存储器并行环境：pthread、cilk、CUDA、OpenCL.
- 支持显式消息传递并行环境：MPI、PVM

消息传递并行程序设计

- 消息传递并行程序设计
  - 用户必须通过显式地发送和接收消息来实现处理机间的数据交换
  - 每个并行进程均有自己独立的地址空间，相互之间访问不能直接进行，必须通过显式的消息传递来实现
  - 适用于大规模并行处理机（MPP）和机群（Cluster）
- 并行计算粒度大，适合大规模可扩展并行算法
  - 消息传递程序设计要求用户很好地分解问题,组织不同进程间的数据交换,并行计算粒度大,特别适合于大规模可扩展并行算法

什么是MPI?

- Massage Passing Interface:是消息传递函数库的标准规范，由MPI论坛开发，支持Fortran和C
  - 一种新的库描述，不是一种语言
  - 共有上百个函数调用接口，在Fortran和C语言中可以直接对这些函数进行调用
  - 是一种标准或规范，而不是特指某一个对它的具体实现
  - MPI是一种消息传递编程模型，并成为这种编程模型的代表和事实上的标准

为什么要使用MPI?

- 高可移植性
  - MPI已在PC机、MS Windows以及所有主要的Unix工作站上和所有主流的并行机上得到实现
  - 使用MPI作消息传递的C或Fortran并行程序可不加改变地在上述平台实现

MPI的发展过程

- 发展的两个阶段
  - 1994年5月完成1.0版
    - 支持C和Fortran77
    - 制定大部分并行功能
  - 1997年4月完成2.0版
    - 动态进程
    - 并行I/O
    - 支持Fortran 90和C++

常用的MPI版本

- **MPICH**
  - 是MPI最流行的非专利实现,由Argonne国家实验室和密西西比州立大学联合开发,具有更好的可移植性
  - 当前最新版本有MPICH 1.2.7和MPICH2 1.03
- **LAMMPI**
  - 美国Indiana 大学Open Systems 实验室实现
  - 当前最新版本7.1.2
- **更多的商业版本MPI**
  - HP-MPI，MS-MPI，……
- **所有的版本遵循MPI标准，MPI程序可以不加修改的运行**

目前主要的MPI实现

MPICH: http://www-unix.mcs.anl.gov/mpi/mpich

Chimp : ftp://ftp.epcc.ed.ac.uk/pub/packages/chimp/release/

Lam: http://www.mpi.nd.edu/lam/

4个功能强大的MPI通信函数

- **MPI_Allgaterv**：数据全收集函数，不同的进程可以提供不同的数量的元素
- **MPI_Scatterv:**散发数据操作，不同的进程可以获得不同数目的元素
- **MPI_Gatherv**：数据收集操作，从不同进程收集来的元素的个数可以不同
- **MPI_Alltoall**：全交换操作，在所有进程间交换数据元素

矩阵向量乘法

- **Sequential algorithm and its complexity**

$$\begin{bmatrix} 2 & 1 & 0 & 4 \\ 3 & 2 & 1 & 1 \\ 4 & 3 & 1 & 2 \\ 3 & 0 & 2 & 0 \end{bmatrix} \times \begin{bmatrix} 1 \\ 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 9 \\ 14 \\ 19 \\ 11 \end{bmatrix}$$

矩阵向量乘法

输入 ：$a[0..m-1,0..n-1]$——$m\times n$ 的矩阵

$b[0..n-1]$——n 维向量

输出: $c[0..m-1]$——m 维向量

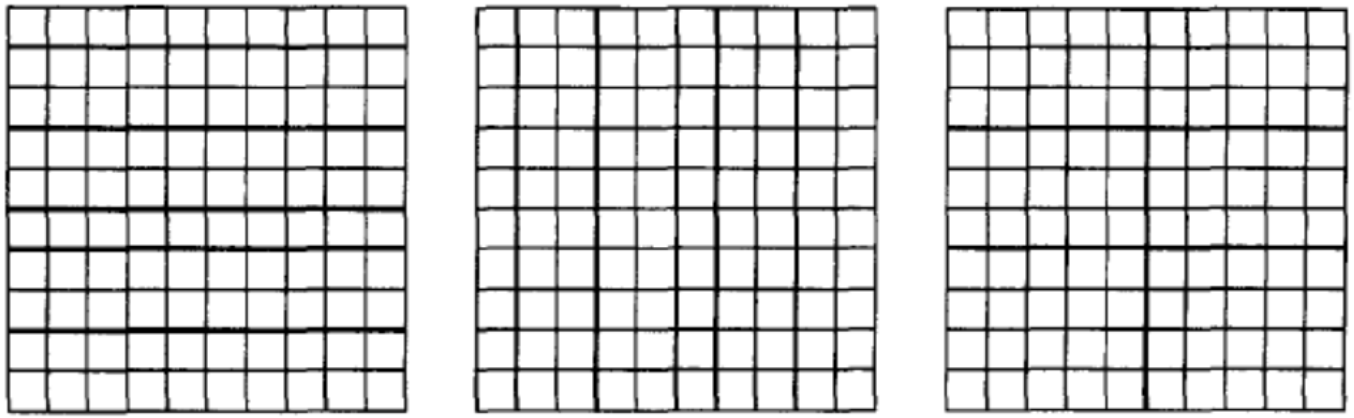
```
for i ← 0 to m − 1
    c[i] ← 0
    for j ← 0 to n − 1
        c[i] ← c[i] + a[i, j] × b[j]
    endfor
endfor
```

矩阵向量乘法需要完成m个内积计算，所以算法复杂度O(mn).

矩阵向量乘法

- **Design, analysis, and implementation of three parallel programs**
  - Rowwise block striped（按行分解）
  - Columnwise block striped（按列分解）
  - Checkerboard block（按块分解）

矩阵的三种分解方式



（a）　　（b）　　（c）

矩阵分解三种方式(a)按行分解　（b）按列分解　（c）棋盘式分解

向量b 和c , 有两种分解方式

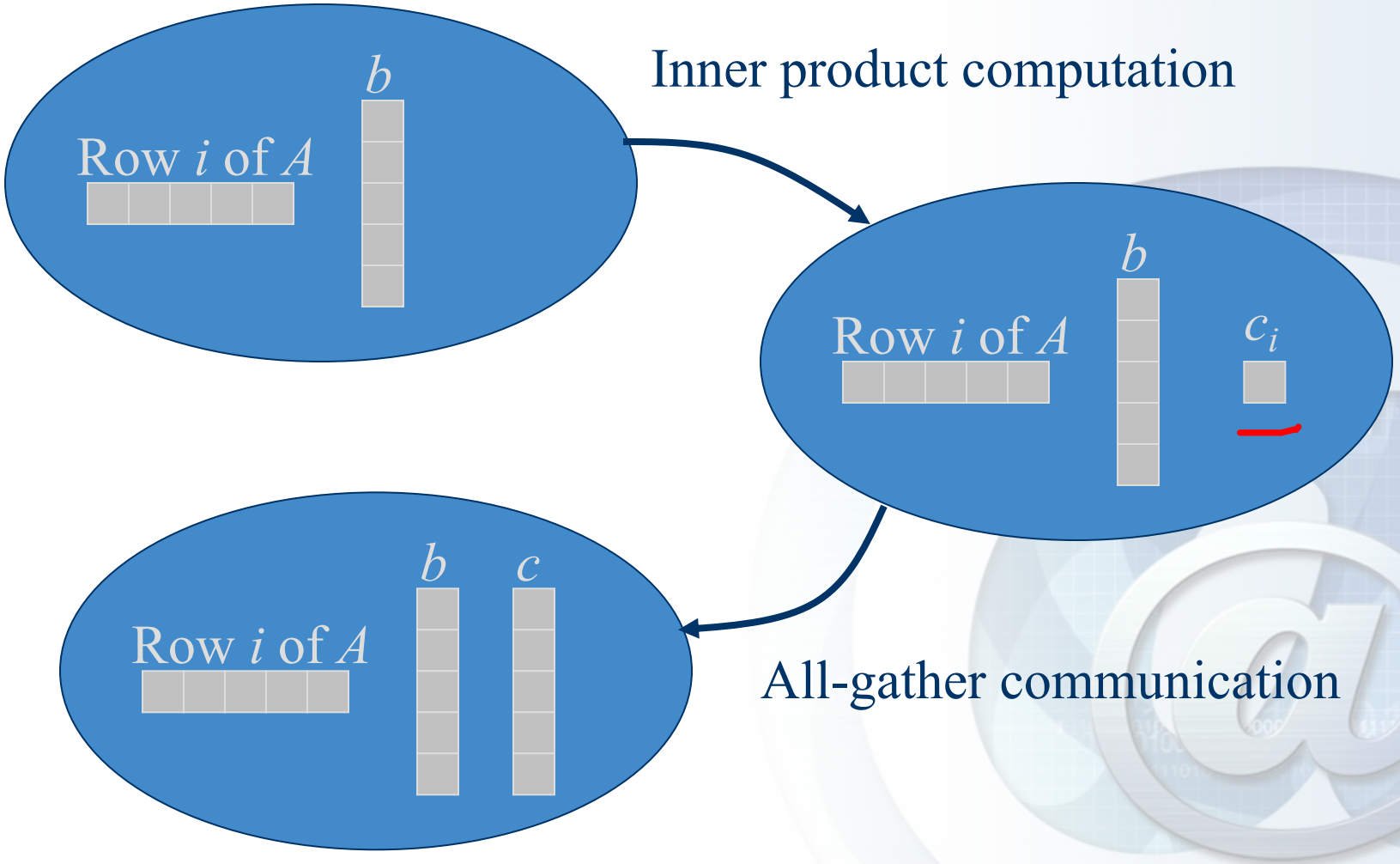
Storing Vectors

- Divide vector elements among processes
- Replicate vector elements
- Vector replication acceptable because vectors have only *n* elements, versus n^2 elements in matrices

矩阵按行分解

- **Partitioning through domain decomposition**
- **Primitive task associated with**
  - Row of matrix
  - Entire vector

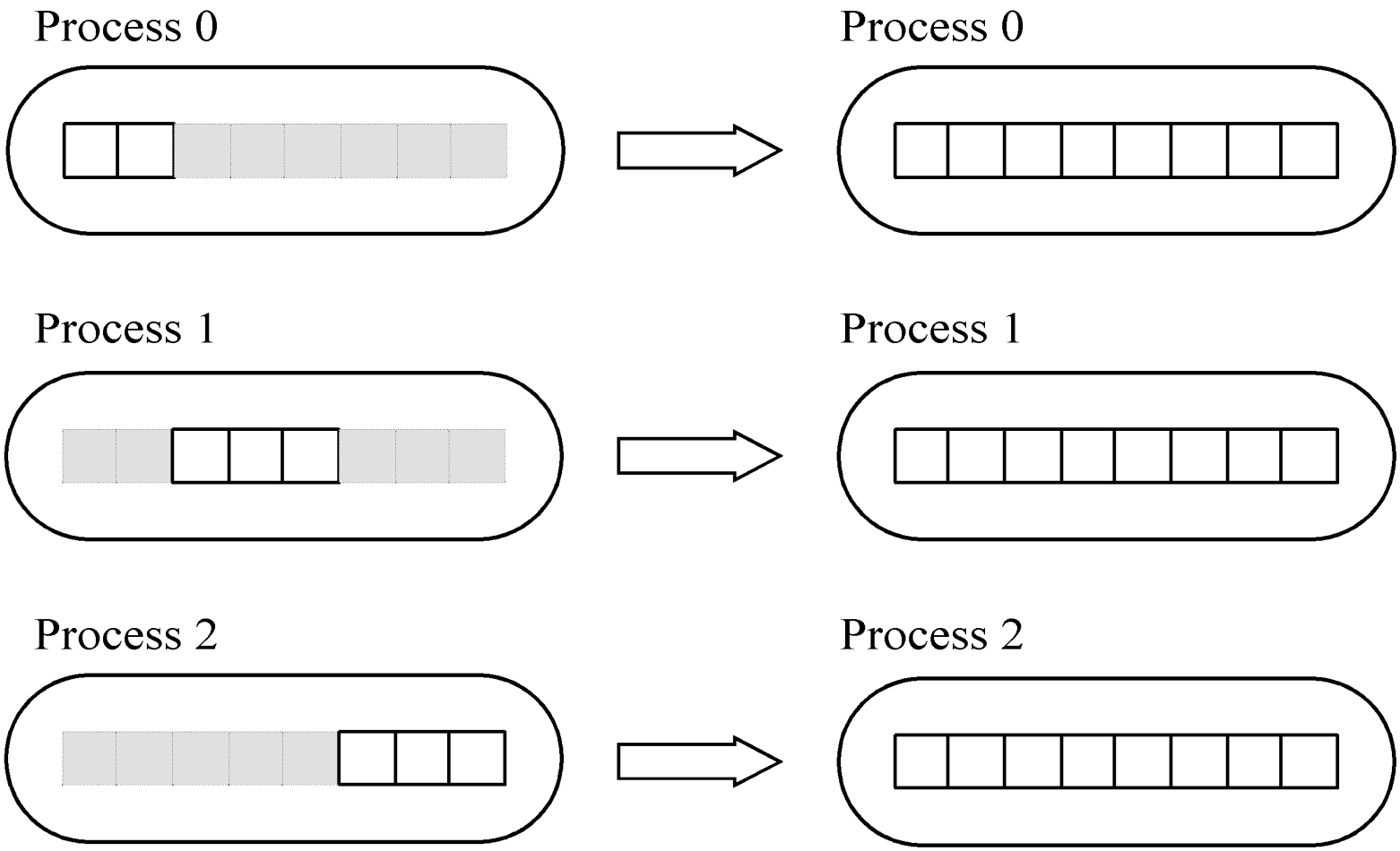
Phases of Parallel Algorithm



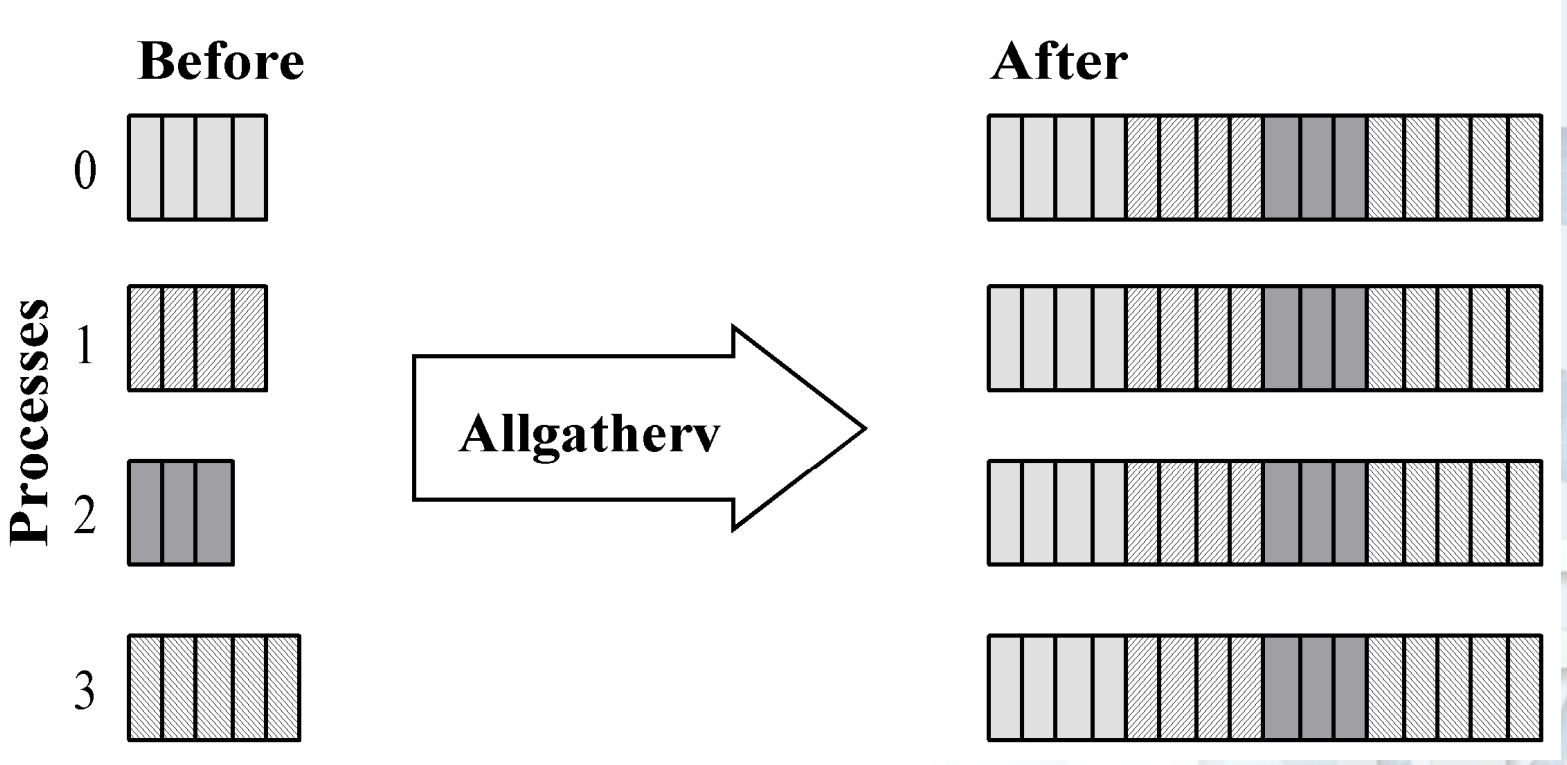
复制分块向量

- Create space for entire vector
- Create “mixed transfer” arrays
- Call `MPI_Allgatherv`

复制分块向量



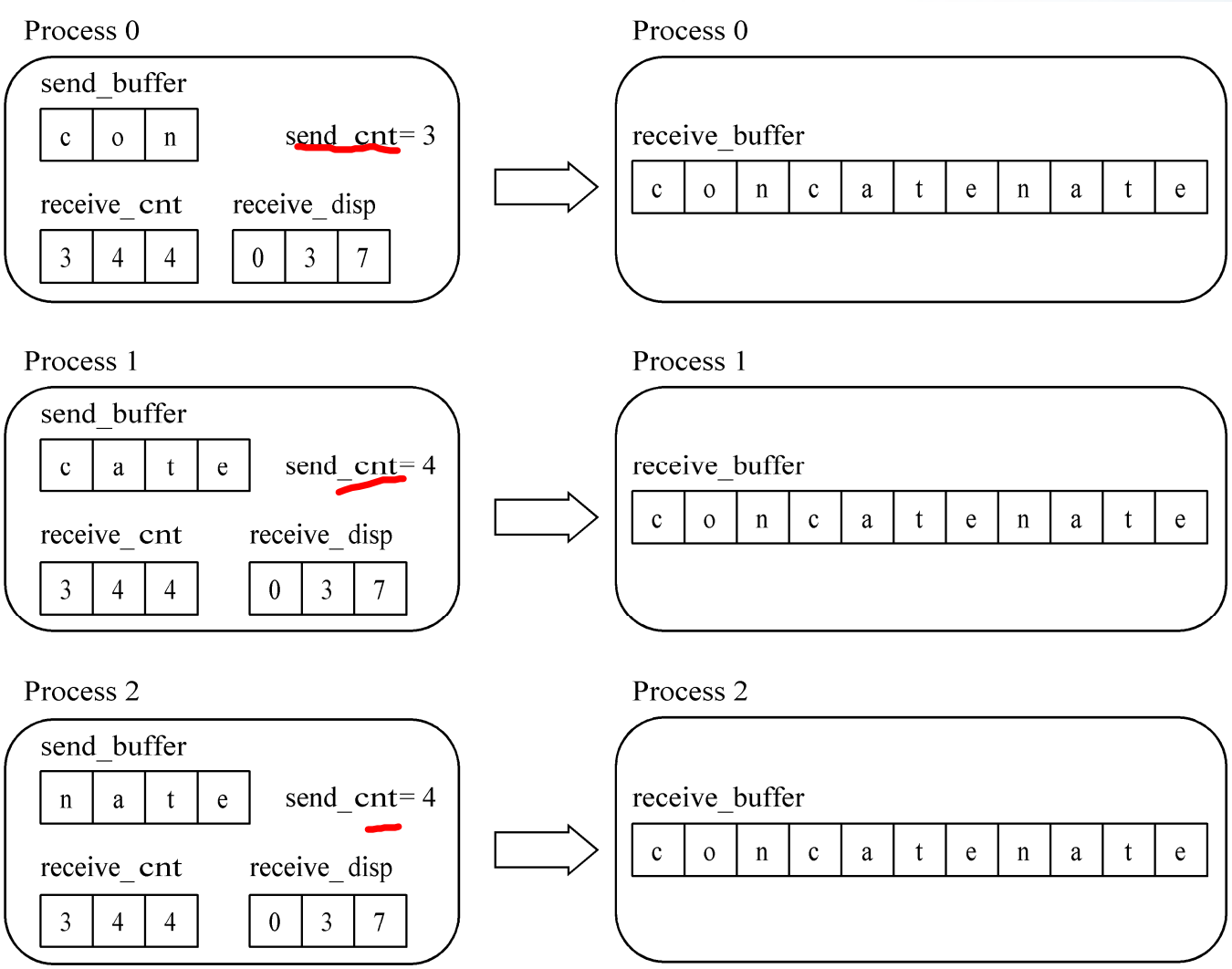
MPI_Allgatherv



MPI_Allgatherv

```
int MPI_Allgatherv (
    void            *send_buffer,
    int              send_cnt,
    MPI_Datatype     send_type,
    void            *receive_buffer,
    int             *receive_cnt,
    int             *receive_disp,
    MPI_Datatype     receive_type,
    MPI_Comm         communicator)
```

MPI_Allgatherv in Action



Agglomeration and Mapping

- **Static number of tasks**
- **Regular communication pattern (all-gather)**
- **Computation time per task is constant**
- **Strategy:**
  - Agglomerate groups of rows
  - Create one task per MPI process

Function read_replicated_vector

- **Process *p*-1**
  - Opens file
  - Reads vector length
- **Broadcast vector length (root process = *p*-1)**
- **Allocate space for vector**
- **Process *p*-1 reads vector, closes file**
- **Broadcast vector**

Function print_replicated_vector

- Process 0 prints vector
- Exact call to `printf` depends on value of parameter `datatype`

矩阵按列分解

- **Partitioning through domain decomposition**
- **Task associated with**
  - Column of matrix
  - Vector element

Matrix-Vector Multiplication

$$
\begin{aligned}
c_0 &= a_{0,0}\, b_0 + a_{0,1}\, b_1 + a_{0,2}\, b_2 + a_{0,3}\, b_3 + a_{4,4}\, b_4 \\
c_1 &= a_{1,0}\, b_0 + a_{1,1}\, b_1 + a_{1,2}\, b_2 + a_{1,3}\, b_3 + a_{1,4}\, b_4 \\
c_2 &= a_{2,0}\, b_0 + a_{2,1}\, b_1 + a_{2,2}\, b_2 + a_{2,3}\, b_3 + a_{2,4}\, b_4 \\
c_3 &= a_{3,0}\, b_0 + a_{3,1}\, b_1 + a_{3,2}\, b_2 + a_{3,3}\, b_3 + b_{3,4}\, b_4 \\
c_4 &= a_{4,0}\, b_0 + a_{4,1}\, b_1 + a_{4,2}\, b_2 + a_{4,3}\, b_3 + a_{4,4}\, b_4
\end{aligned}
$$

Proc 4

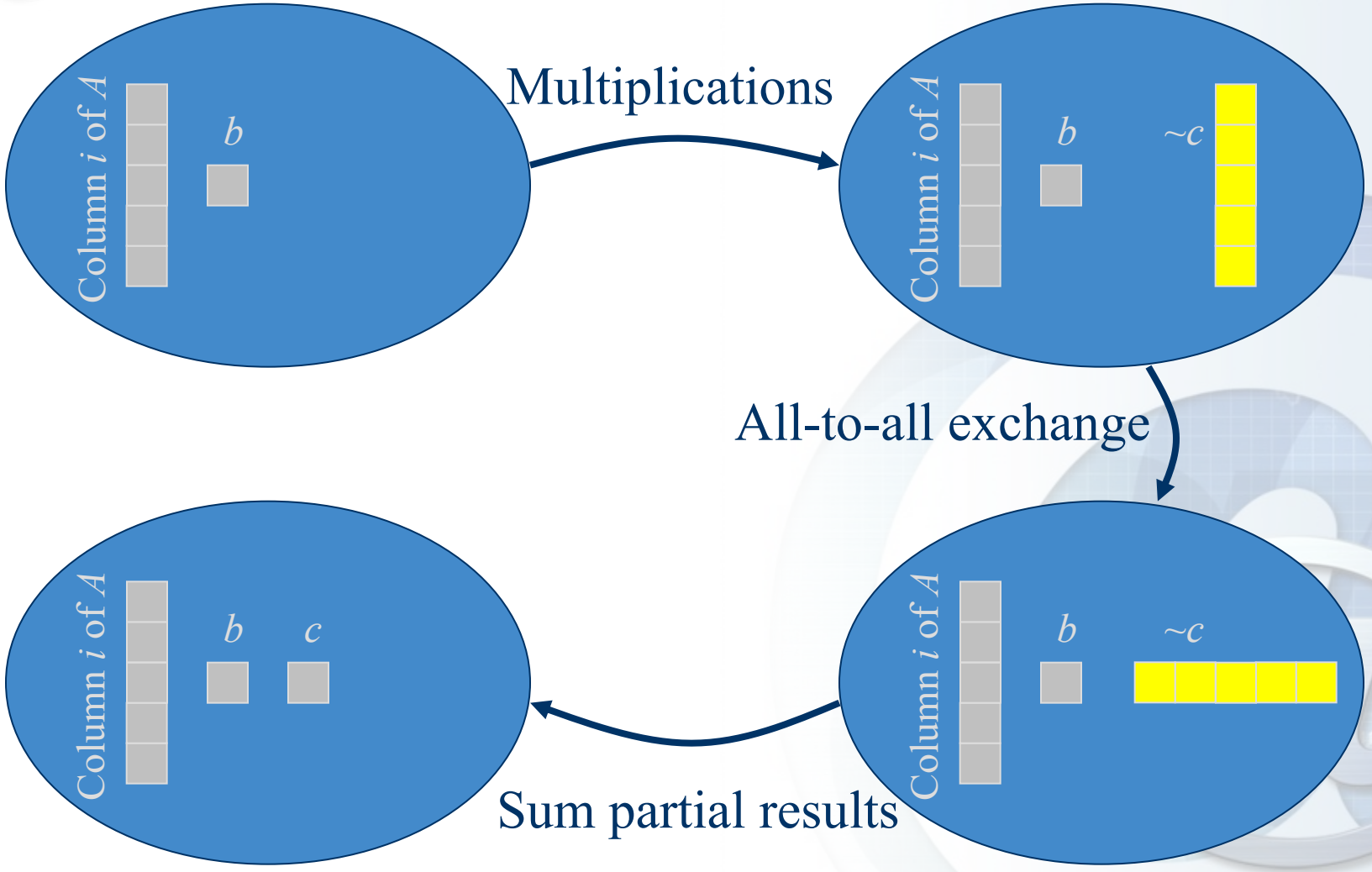
Proc 3

Proc 2

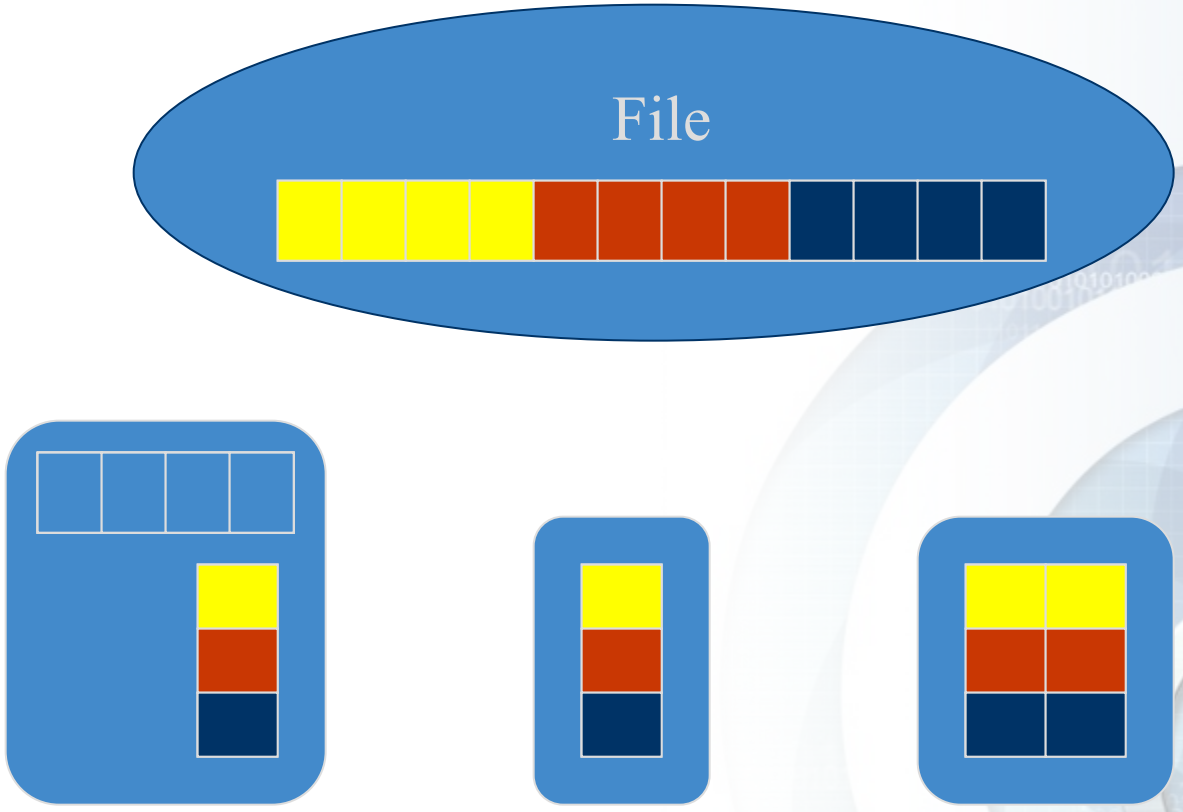
Processor 1's initial computation

Processor 0's initial computation

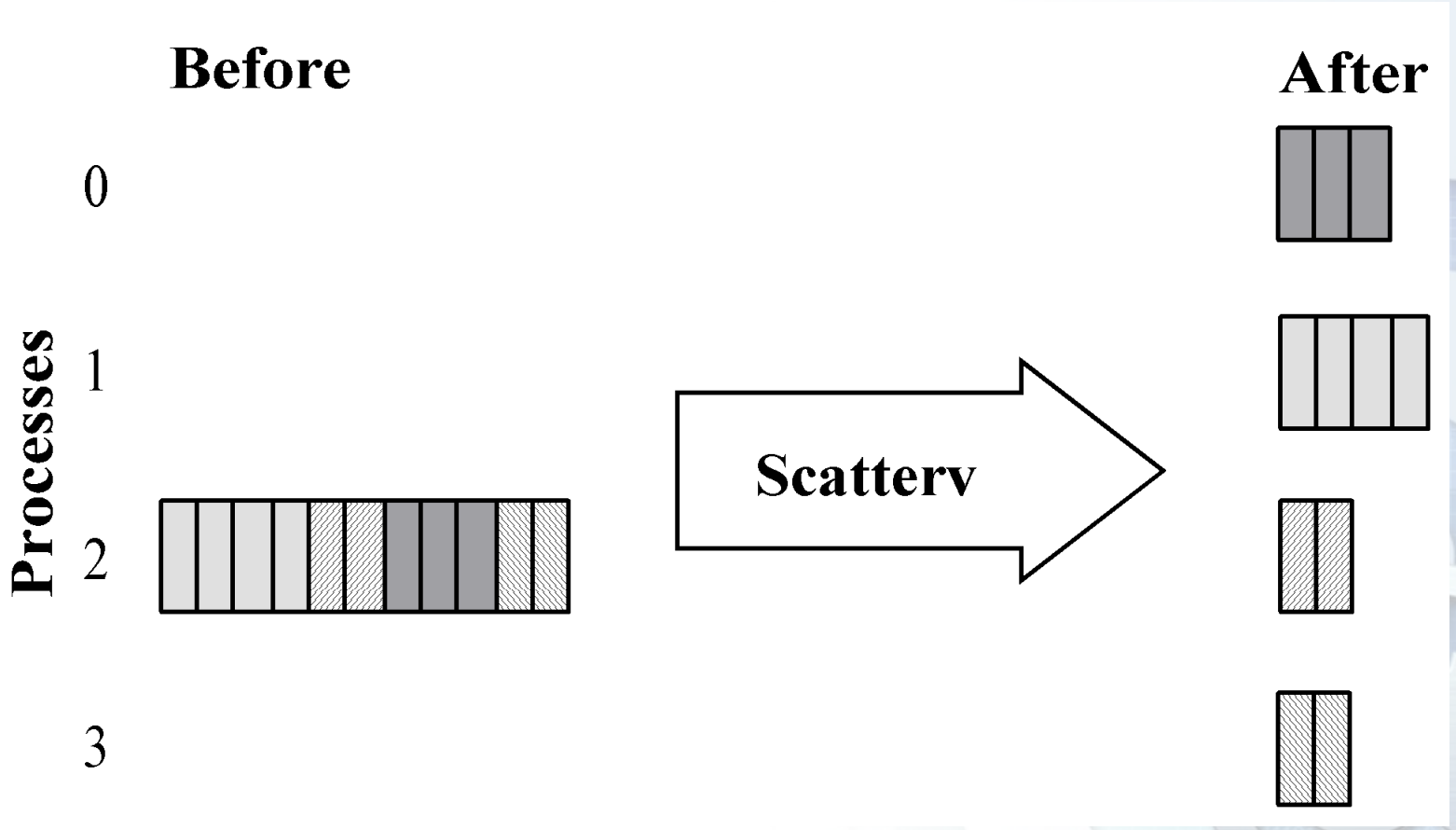
Phases of Parallel Algorithm



读取按列分解的矩阵



MPI_Scatterv



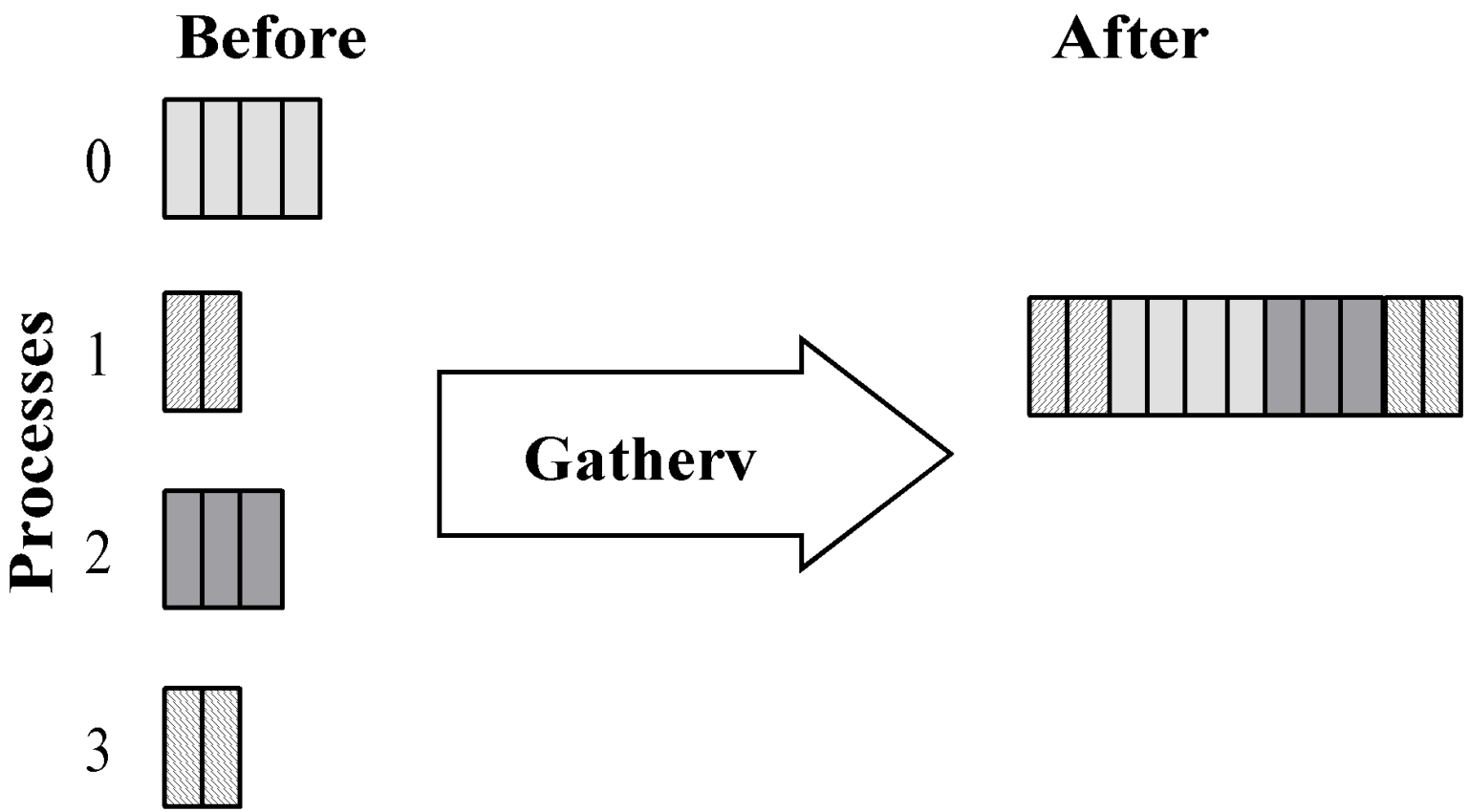
通过一个根进程向在一个通信域内的所有进程（包括自身）分发一组连续的数据元素

Header for MPI_Scatterv

```
int MPI_Scatterv (
    void            *send_buffer,
    int             *send_cnt,
    int             *send_disp,
    MPI_Datatype     send_type,
    void            *receive_buffer,
    int              receive_cnt,
    MPI_Datatype     receive_type,
    int              root,
    MPI_Comm         communicator)
```

MPI_Scatterv 是一组通信函数，通信域中所有进程都参与其执行。

Function MPI_Gatherv



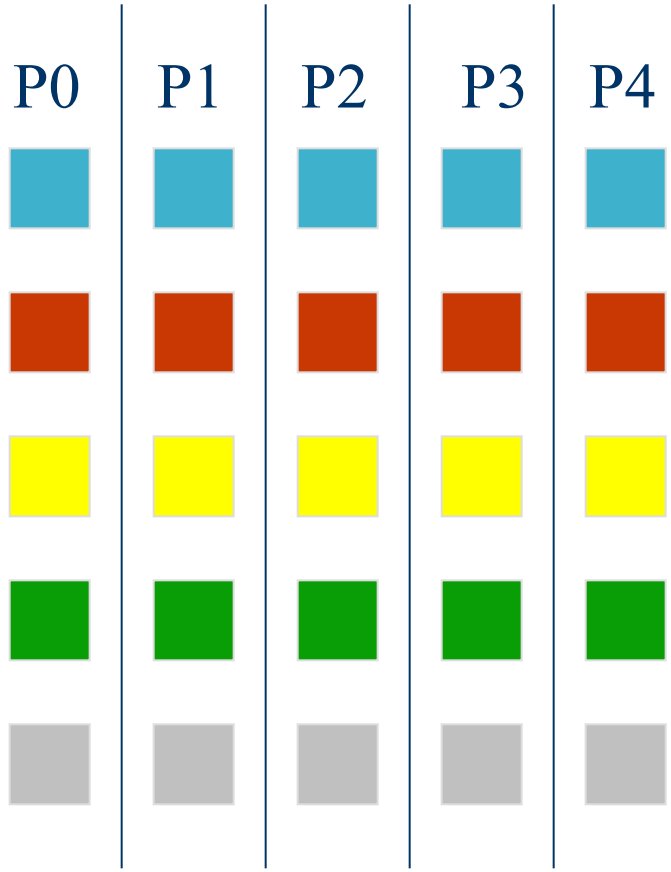
将存储在通信域所有进程中数据收集到某个进程中

Header for MPI_Gatherv

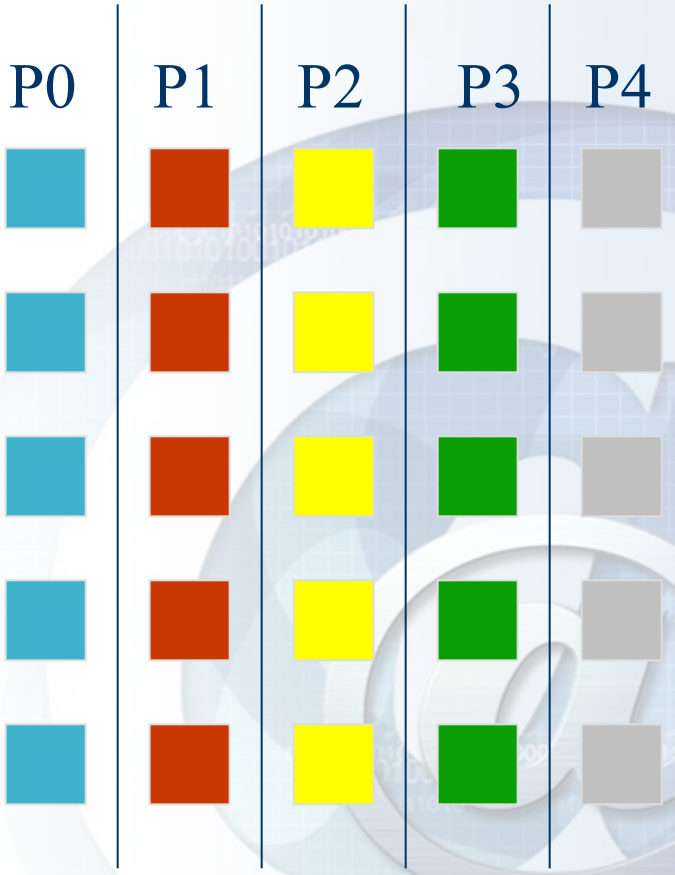
```
int MPI_Gatherv (
    void            *send_buffer,
    int              send_cnt,
    MPI_Datatype     send_type,
    void            *receive_buffer,
    int             *receive_cnt,
    int             *receive_disp,
    MPI_Datatype     receive_type,
    int              root,
    MPI_Comm         communicator)
```

将存储在通信域所有进程中数据收集到某个进程中

All-to-all Exchange



(Before)



(After)

Header for MPI_Alltoallv

```
int MPI_Alltoallv (
    void            *send_buffer,
    int             *send_cnt,
    int             *send_disp,
    MPI_Datatype     send_type,
    void            *receive_buffer,
    int             *receive_cnt,
    int             *receive_disp,
    MPI_Datatype     receive_type,
    MPI_Comm         communicator)
```

完成一个通信域的所有进程之间相互交换数据

Agglomeration and Mapping

- **Static number of tasks**
- **Regular communication pattern (all-to-all)**
- **Computation time per task is constant**
- **Strategy:**
  - Agglomerate groups of columns
  - Create one task per MPI process

Printing a Block-Column Matrix

- Data motion opposite to that we did when reading the matrix
- Replace “scatter” with “gather”
- Use “v” variant because different processes contribute different numbers of elements

Count/Displacement Arrays

- **MPI_Alltoallv requires two pairs of count/displacemen**

  `create_mixed_xfer_arrays` builds these

- **First pair for value**
  - send_cnt: number of elements
  - send_disp: index of first element
- **Second pair for values being received**
  - recv_cnt: number of elements
  - recv_disp: index of first element

Function create_uniform_xfer_arrays

- **First array**
  - How many elements received from each process (always same value)
  - Uses ID and utility macro block_size
- **Second array**
  - Starting position of each process' block
  - Assume blocks in process rank order

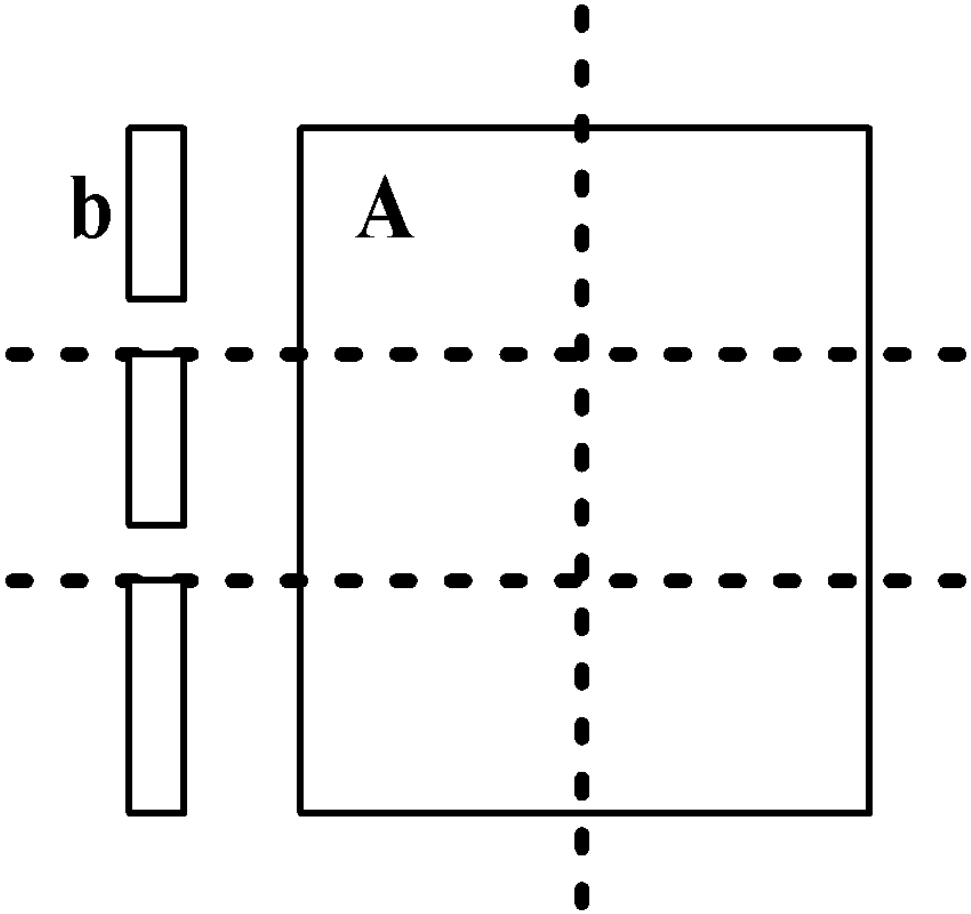
Benchmarking Results

	Execution Time (msec)			
p	*Predicted*	*Actual*	*Speedup*	*Mflops*
1	63.4	63.8	1.00	31.4
2	32.4	32.9	1.92	60.8
3	22.2	22.6	2.80	88.5
4	17.2	17.5	3.62	114.3
5	14.3	14.5	4.37	137.9
6	12.5	12.6	5.02	158.7
7	11.3	11.2	5.65	178.6
8	10.4	10.0	6.33	200.0
16	8.5	7.6	8.33	263.2

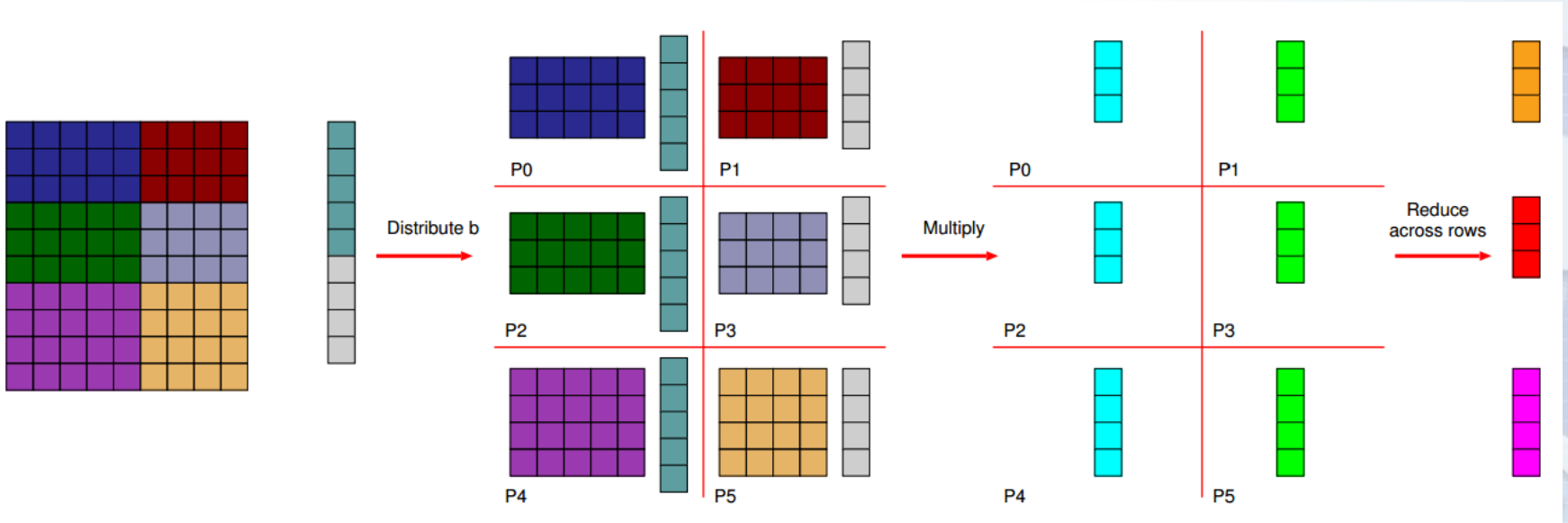
棋盘分解

- Associate primitive task with each element of the matrix A
- Each primitive task performs one multiply
- Agglomerate primitive tasks into rectangular blocks
- Processes form a 2-D grid
- Vector b distributed by blocks among processes in first column of grid

Tasks after Agglomeration



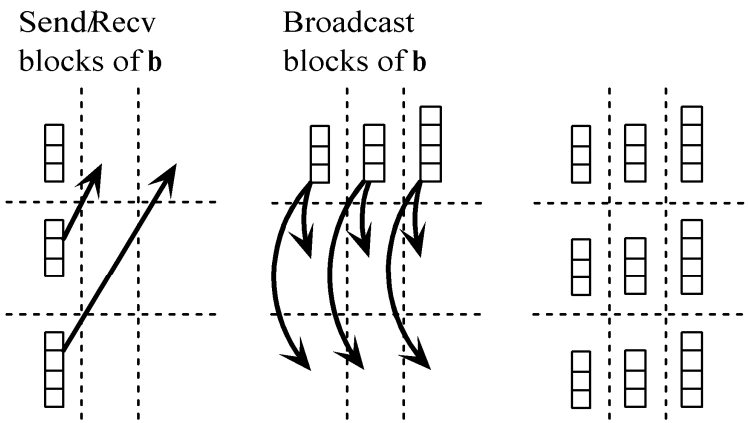
Algorithm's Phases



Redistributing Vector b

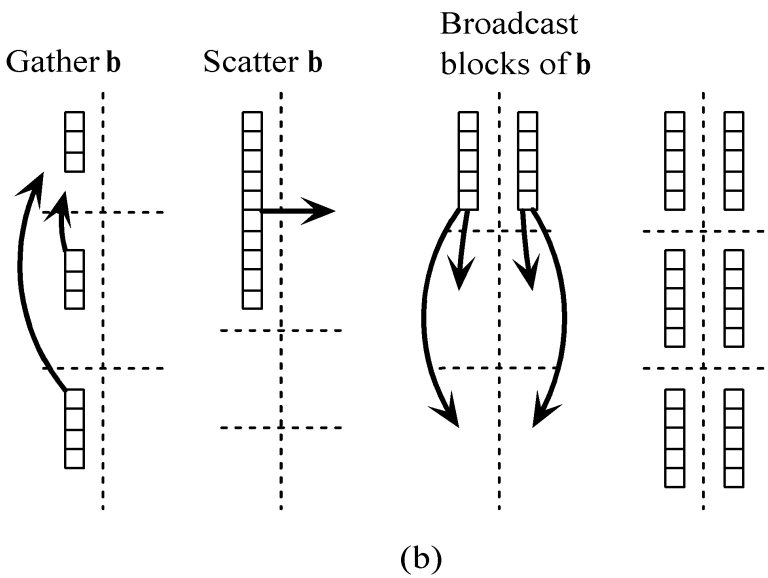
- **Step 1: Move b from processes in first row to processes in first column**
  - If *p* square
    - First column/first row processes send/receive portions of **b**
  - If *p* not square
    - Gather **b** on process 0, 0
    - Process 0, 0 scatter to first row procs
- **Step 2: First row processes broadcast b within columns**

Redistributing Vector b



(a)

When p is a square number



(b)

When p is not a square number

Creating Communicators

- Want processes in a virtual 2-D grid
- Create a custom communicator to do this
- Collective communications involve all processes in a communicator
- We need to do broadcasts, reductions among subsets of processes
- We will create communicators for processes in same row or same column

What's in a Communicator?

- **Process group**
- **Context**
- **Attributes**
  - Topology (lets us address processes another way)
  - Others we won't consider

Creating 2-D Virtual Grid of Processes

- **MPI_Dims_create**
  - Input parameters
    - Total number of processes in desired grid
    - Number of grid dimensions
  - Returns number of processes in each dim
- **MPI_Cart_create**
  - Creates communicator with cartesian topology

MPI_Dims_create

```
int MPI_Dims_create (
    int nodes,
        /* Input - Procs in grid */

    int dims,
        /* Input - Number of dims */

    int *size)
        /* Input/Output - Size of
           each grid dimension */
```

MPI_Cart_create

```
int MPI_Cart_create (
   MPI_Comm old_comm, /* Input - old communicator */

   int dims, /* Input - grid dimensions */

   int *size, /* Input - # procs in each dim */

   int *periodic,
       /* Input - periodic[j] is 1 if dimension j
          wraps around; 0 otherwise */

   int reorder,
       /* 1 if process ranks can be reordered */

   MPI_Comm *cart_comm)
       /* Output - new communicator */
```

Using MPI_Dims_create and MPI_Cart_create

```
MPI_Comm cart_comm;
int p;
int periodic[2];
int size[2];
...
size[0] = size[1] = 0;
MPI_Dims_create (p, 2, size);
periodic[0] = periodic[1] = 0;
MPI_Cart_create (MPI_COMM_WORLD, 2, size,
                 1, &cart_comm);
```

Useful Grid-related Functions

- **MPI_Cart_rank**
  - Given coordinates of process in Cartesian communicator, returns process rank
- **MPI_Cart_coords**
  - Given rank of process in Cartesian communicator, returns process' coordinates

Header for MPI_Cart_rank

```
int MPI_Cart_rank (
    MPI_Comm comm,
        /* In - Communicator */
    int *coords,
        /* In - Array containing process'
                grid location */
    int *rank)
        /* Out - Rank of process at
           specified coords */
```

Header for MPI_Cart_coords

```
int MPI_Cart_coords (
    MPI_Comm comm,
        /* In - Communicator */
    int rank,
        /* In - Rank of process */
    int dims,
        /* In - Dimensions in virtual grid */
    int *coords)
        /* Out - Coordinates of specified
           process in virtual grid */
```

MPI_Comm_split

- Partitions the processes of a communicator into one or more subgroups
- Constructs a communicator for each subgroup
- Allows processes in each subgroup to perform their own collective communications
- Needed for columnwise scatter and rowwise reduce

Header for MPI_Comm_split

```
int MPI_Comm_split (
   MPI_Comm old_comm,
      /* In - Existing communicator */

   int partition, /* In - Partition number */

   int new_rank,
      /* In - Ranking order of processes
         in new communicator */

   MPI_Comm *new_comm)
      /* Out - New communicator shared by
         processes in same partition */
```

Example: Create Communicators for Process Rows

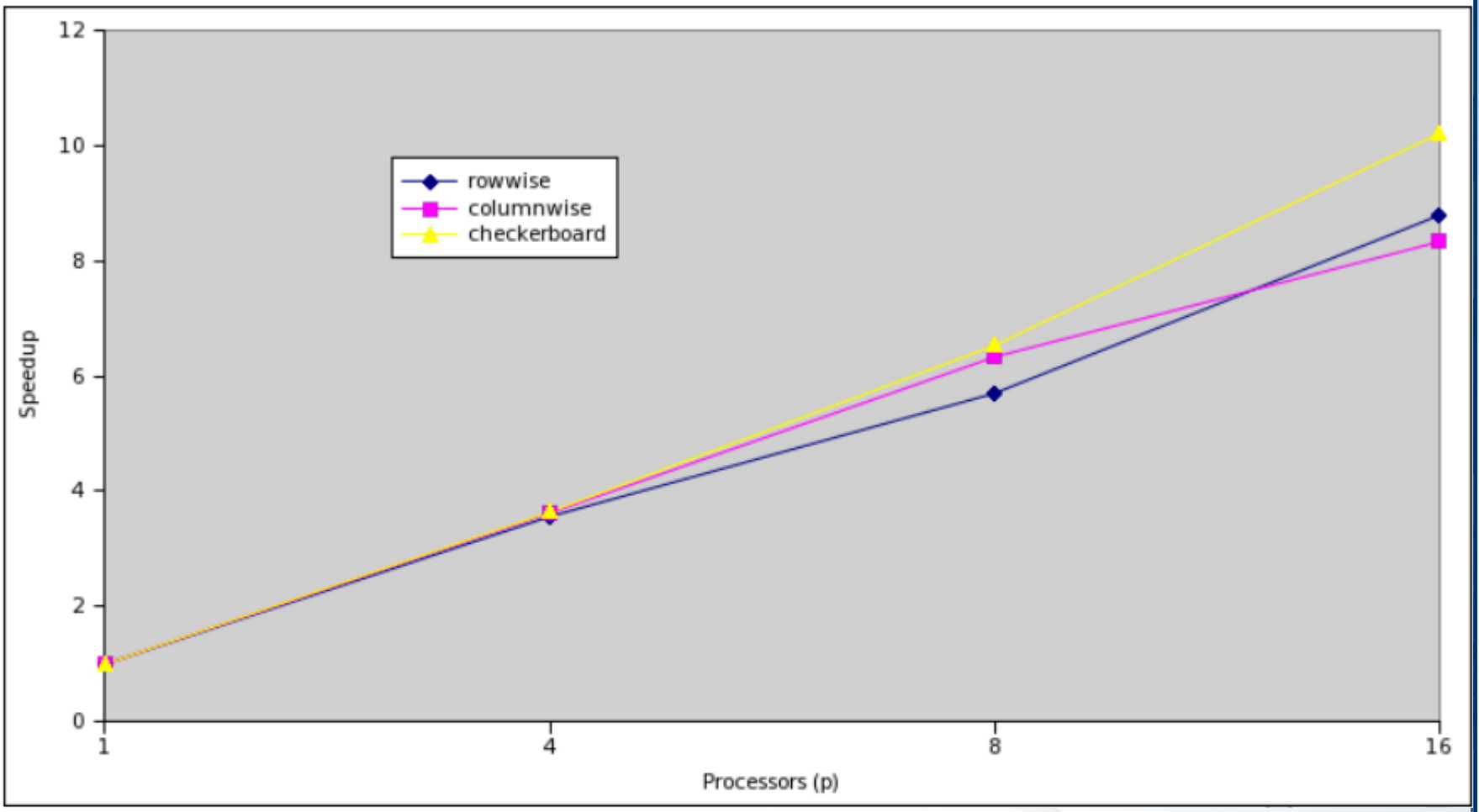
```
MPI_Comm grid_comm; /* 2-D process grid */

MPI_Comm grid_coords[2];
         /* Location of process in grid */

MPI_Comm row_comm;
               /* Processes in same row */

MPI_Comm_split (grid_comm, grid_coords[0],
   grid_coords[1], &row_comm);
```

Comparison of Three Algorithms



Summary (1/3)

- **Matrix decomposition $\Rightarrow$ communications needed**
  - Rowwise block striped: all-gather
  - Columnwise block striped: all-to-all exchange
  - Checkerboard block: gather, scatter, broadcast, reduce
- **All three algorithms: roughly same number of messages**
- **Elements transmitted per process varies**
  - First two algorithms: $\Theta(n)$ elements per process
  - Checkerboard algorithm: $\Theta(n/\sqrt{p})$ elements
- **Checkerboard block algorithm has better scalability**

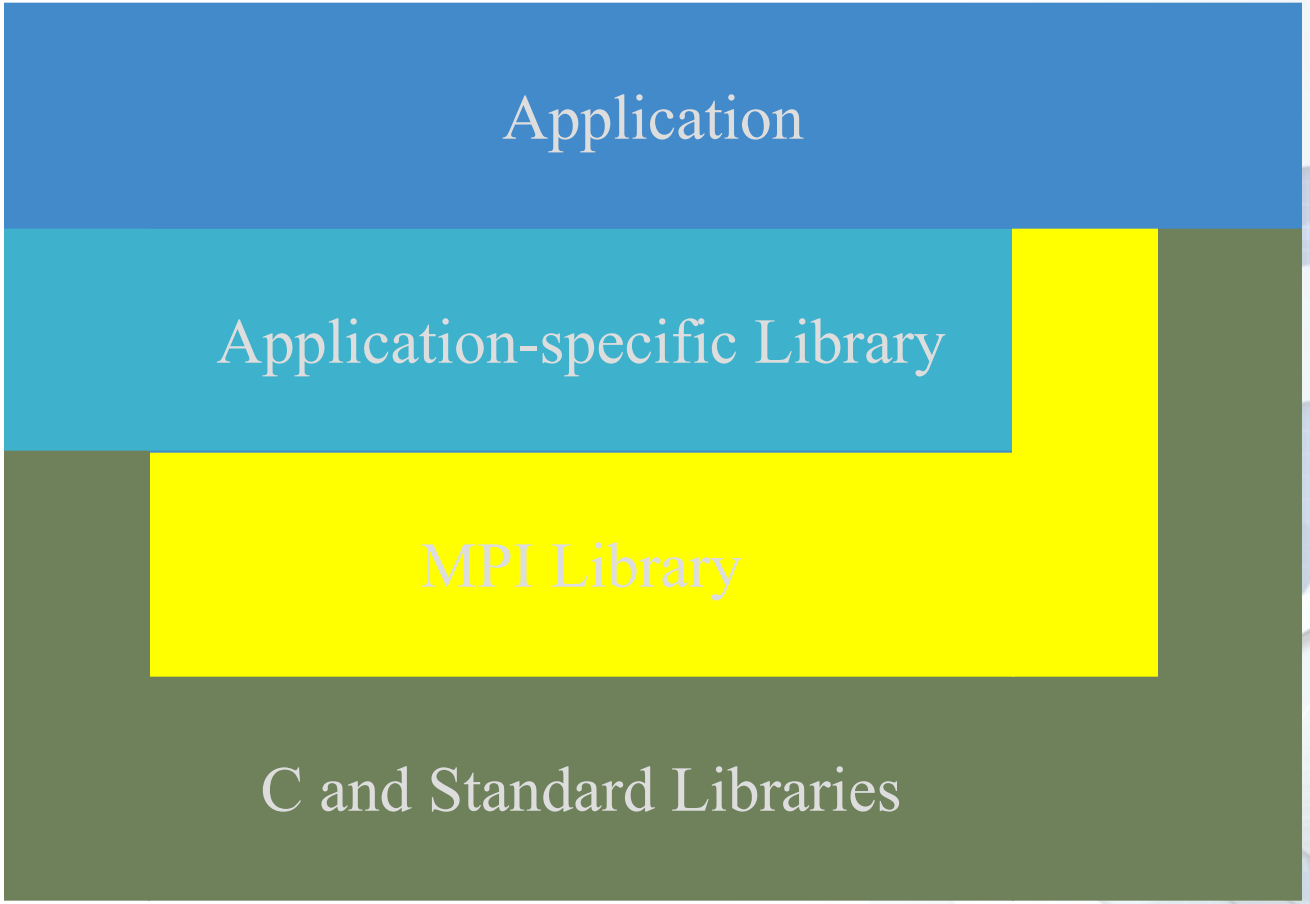
Summary (2/3)

- **Communicators with Cartesian topology**
  - Creation
  - Identifying processes by rank or coords
- **Subdividing communicators**
  - Allows collective operations among subsets of processes

Summary (3/3)

- Parallel programs and supporting functions much longer than C counterparts
- Extra code devoted to reading, distributing, printing matrices and vectors
- Developing and debugging these functions is tedious and difficult
- Makes sense to generalize functions and put them in libraries for reuse

MPI Application Development



GPU介绍

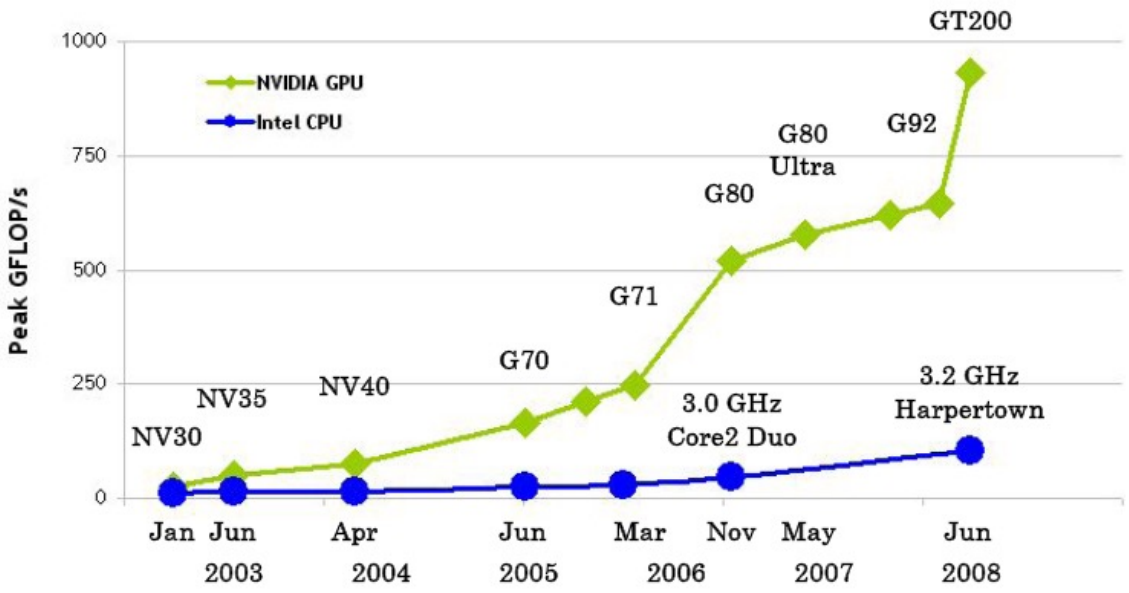
GPU英文全称Graphic Processing Unit，译为“图形处理器”。GPU是相对于CPU的一个概念，GPU是显示卡的“心脏”，也就相当于CPU在电脑中的作用，它决定了该显卡的档次和大部分性能。

GPU的优势

- 强大的处理能力 GPU接近1Tflops/s
- 高带宽 140GB/s
- 低成本 Gflop/$和Gflops/w高于CPU

- 当前世界超级计算机五百强的入门门槛为12Tflops/s
- 一个三节点，每节点4GPU的集群，总处理能力就超过12Tflops/s，如果使用GTX280只需10万元左右，使用专用的Tesla也只需20万左右

GPU /CPU计算能力比较



GT200 = GeForce GTX 280	G71 = GeForce 7900 GTX	NV35 = GeForce FX 5950 Ultra
G92 = GeForce 9800 GTX	G70 = GeForce 7800 GTX	NV30 = GeForce FX 5800
G80 = GeForce 8800 GTX	NV40 = GeForce 6800 Ultra	

GPU/CPU存储器带宽比较

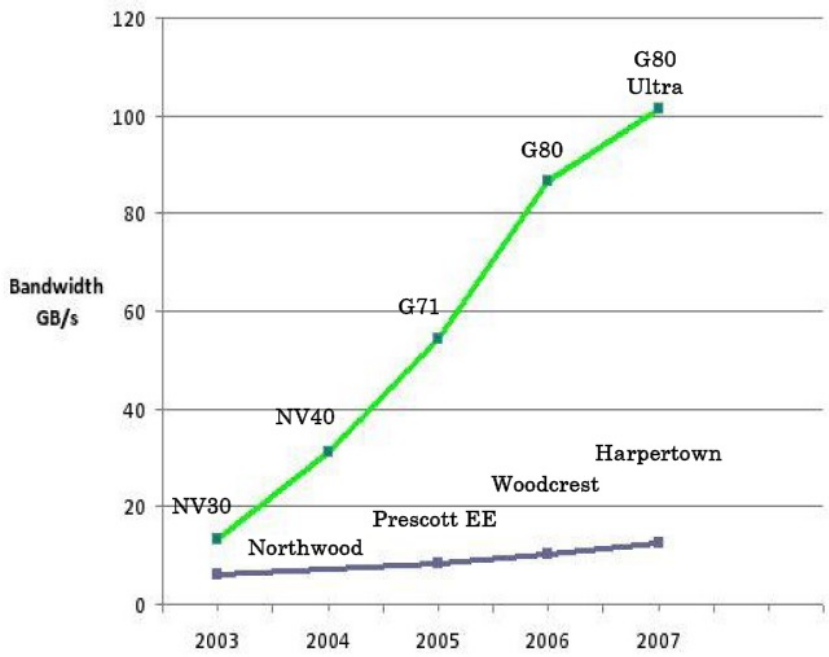
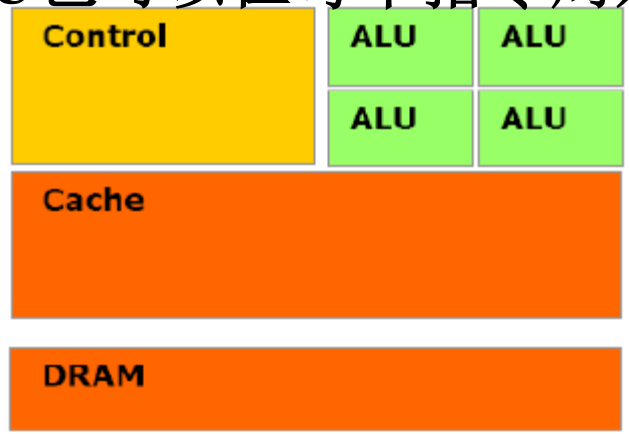


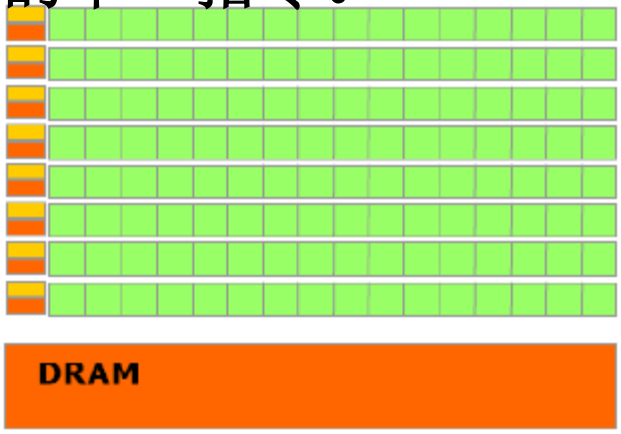
Figure 1-1. Floating-Point Operations per Second and Memory Bandwidth for the CPU and GPU

GPU与CPU的比较

- CPU的内部结构：控制单元、逻辑单元和存储单元
  - ✓运算器：算术运算(如加减乘除)和逻辑运算(如逻辑加、逻辑乘和非运算)
  - ✓控制器：读取各种指令,并对指令进行分析,作出相应的控制。
  - ✓若干个寄存器：可直接参与运算并存放运算的中间结果。
- GPU采用了大量的执行单元，这些执行单元可以轻松的加载并行处理，而不像CPU那样的单线程处理。另外，现代的GPU也可以在每个指令周期执行更多的单一指令。

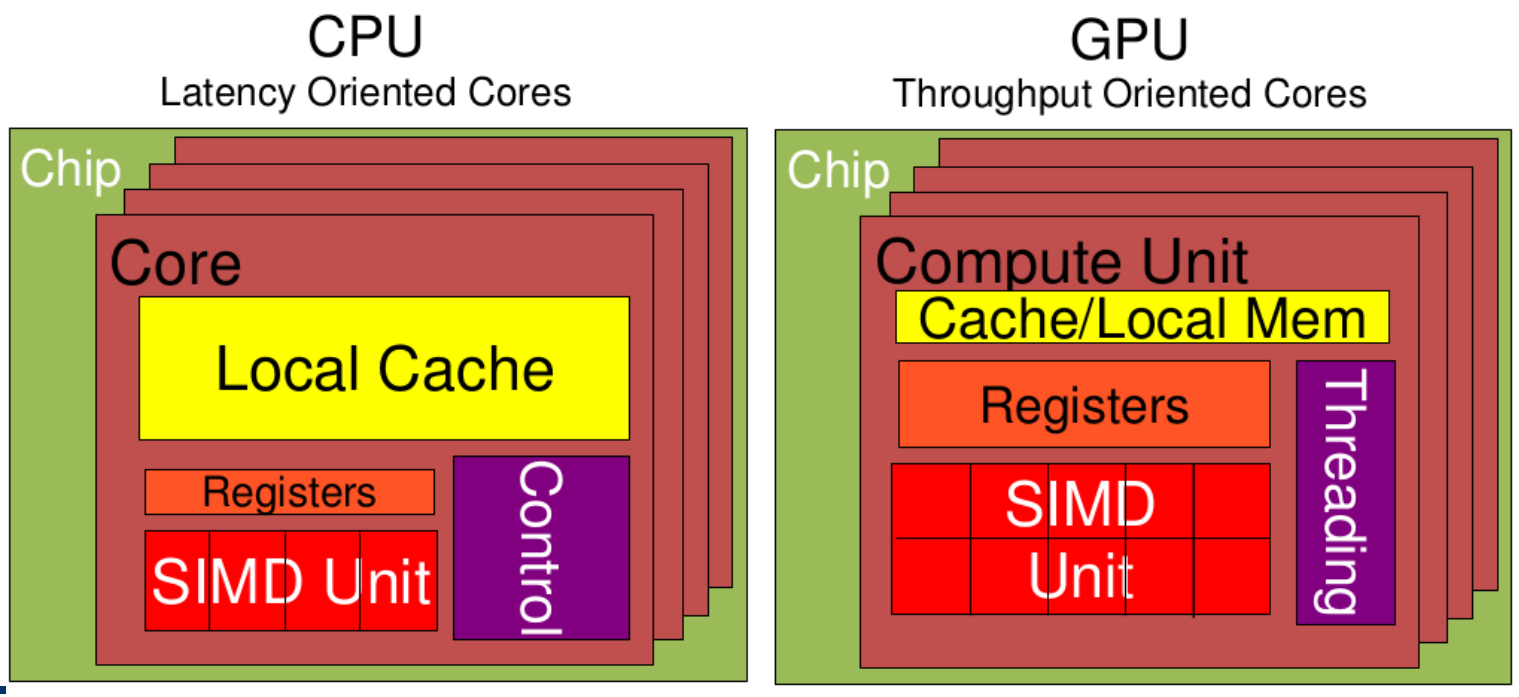


CPU



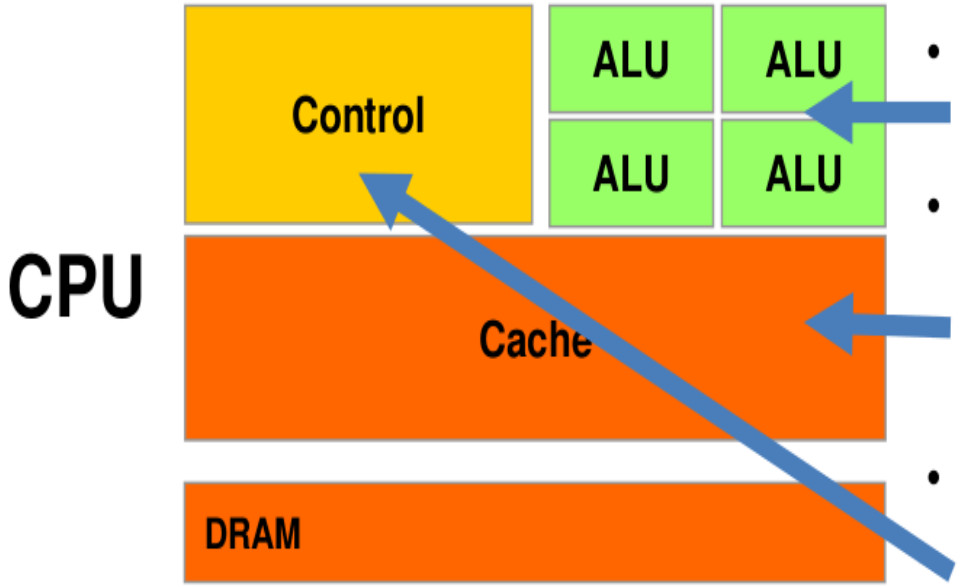
GPU

- Cache, local memory：CPU > GPU
- Threads(线程数)：GPU > CPU
- Registers：GPU > CPU，多寄存器可以支持非常多的Thread，Thread需要用到Register，Thread数目大，Register也必须跟着很大才行。
- SIMD Unit(单指令多数据流，以同步方式，在同一时间内执行同一条指令)：GPU > CPU。



CPU基于低延时的设计

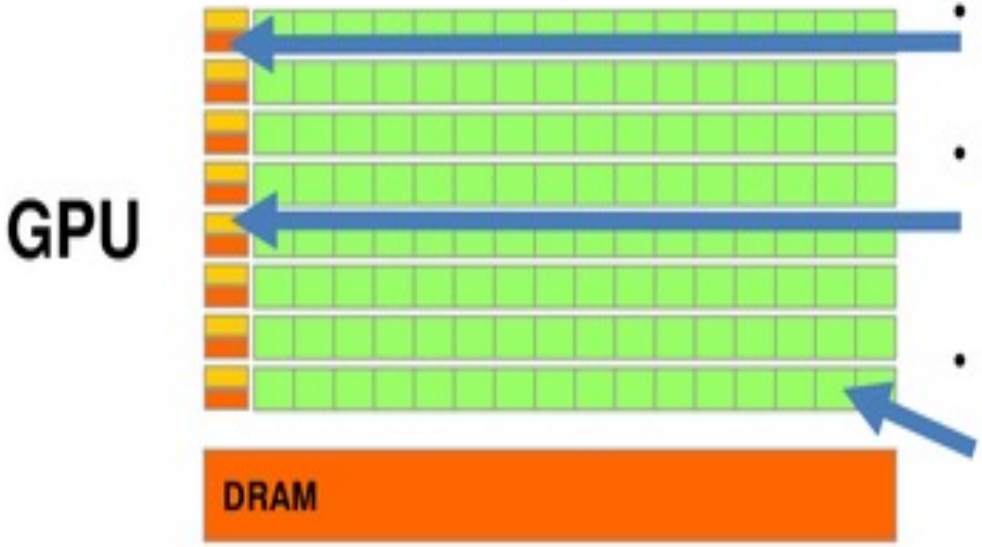
CPUs: Latency Oriented Desigr



- Powerful ALU
  - Reduced operation latency
- Large caches
  - Convert long latency memory accesses to short latency cache accesses
- Sophisticated control
  - Branch prediction for reduced branch latency
  - Data forwarding for reduced data latency

GPU是基于大的吞吐量设计

GPUs: Throughput Oriented Design



- Small caches
  - To boost memory throughput
- Simple control
  - No branch prediction
  - No data forwarding
- Energy efficient ALUs
  - Many, long latency but heavily pipelined for high throughput
- Require massive number of threads to tolerate latencies

GPU发展

- 1993年，黄仁勋与C.P和C.M共同成立NVIDIA公司。
- 2007年，CUDA正式发布，引发GPU通用计算的革命。
- 2009年，GeForce GTS 250发布。
- 2011年3月24日，GeForce GTX 590发布，这是Nvidia公司最新的显卡。

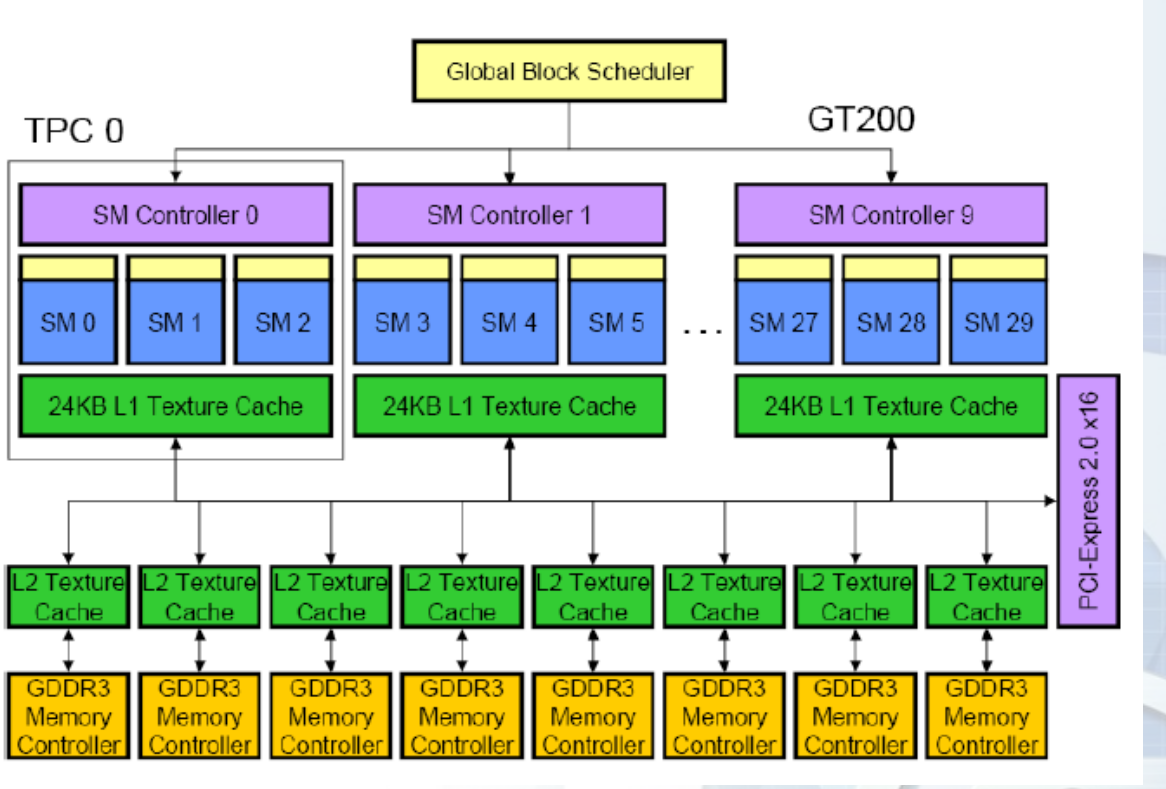
CUDA

- CUDA(Compute Unified Device Architecture，计算统一设备架构)，由显卡厂商Nvidia推出的运算平台。
- CUDA是一种通用并行计算架构，该架构使GPU能够解决复杂的计算问题。开发人员可以使用C语言来为CUDA架构编写程序。
- CUDA的硬件架构
- CUDA的软件架构

体系架构

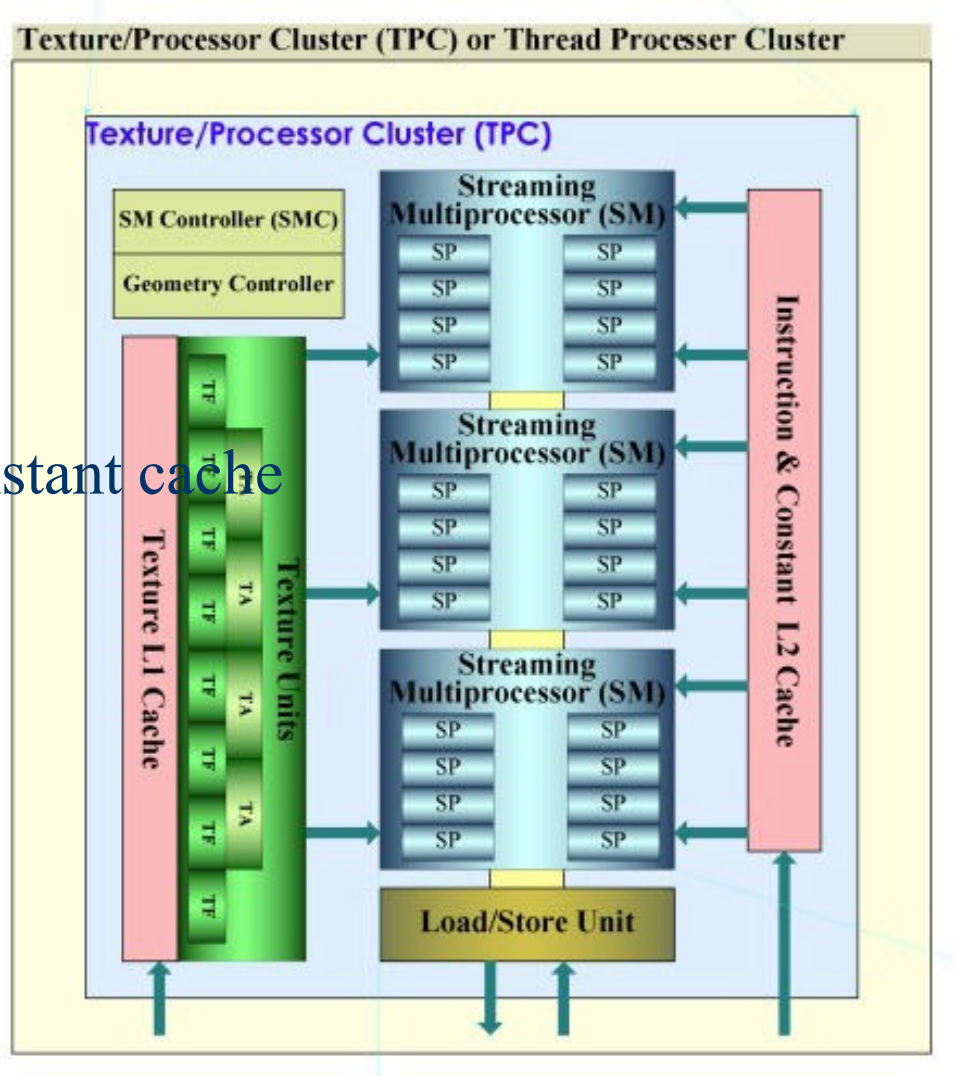
- 体系架构由两部分组成，分别是流处理器阵列（SPA）和存储器系统。（GT200）
- GPU的巨大计算能力来自SPA中的大量计算单元，SPA的结构又分为两层：TPC（线程处理器群）和SM（流多处理器）；
- 存储器系统由几个部分组成：存储器控制器（MMC），固定功能的光栅操作单元（ROP），以及二级纹理缓存。

GT200架构

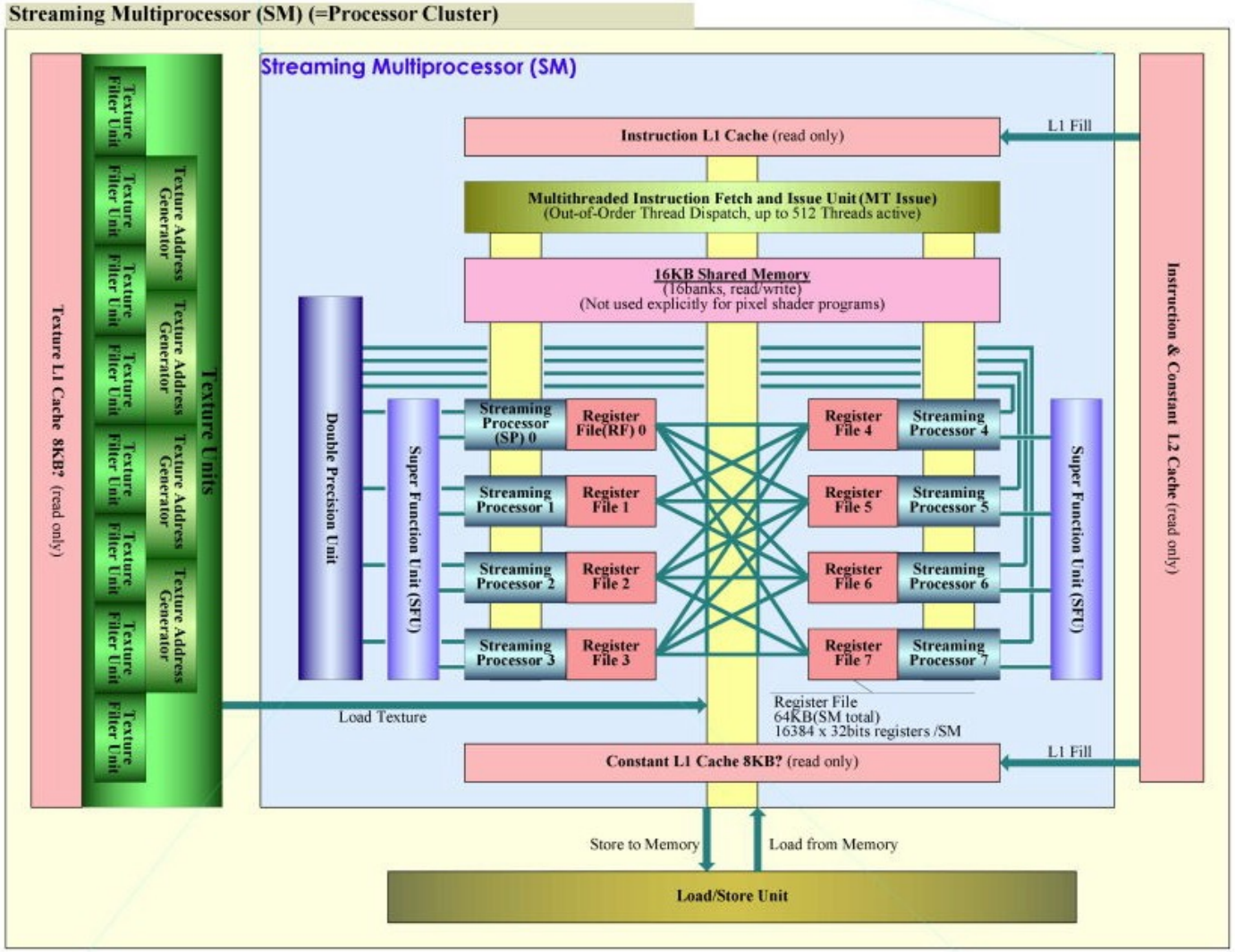

Global Block Scheduler
TPC 0
GT200
SM Controller 0
SM Controller 1
SM Controller 9
SM 0
SM 1
SM 2
SM 3
SM 4
SM 5
. . .
SM 27
SM 28
SM 29
24KB L1 Texture Cache
24KB L1 Texture Cache
24KB L1 Texture Cache
PCI-Express 2.0 x16
L2 Texture Cache
L2 Texture Cache
L2 Texture Cache
L2 Texture Cache
L2 Texture Cache
L2 Texture Cache
L2 Texture Cache
L2 Texture Cache
GDDR3 Memory Controller
GDDR3 Memory Controller
GDDR3 Memory Controller
GDDR3 Memory Controller
GDDR3 Memory Controller
GDDR3 Memory Controller
GDDR3 Memory Controller
GDDR3 Memory Controller

TPC

- 3 SM
- Instruction and constant cache
- Texture
- Load/store

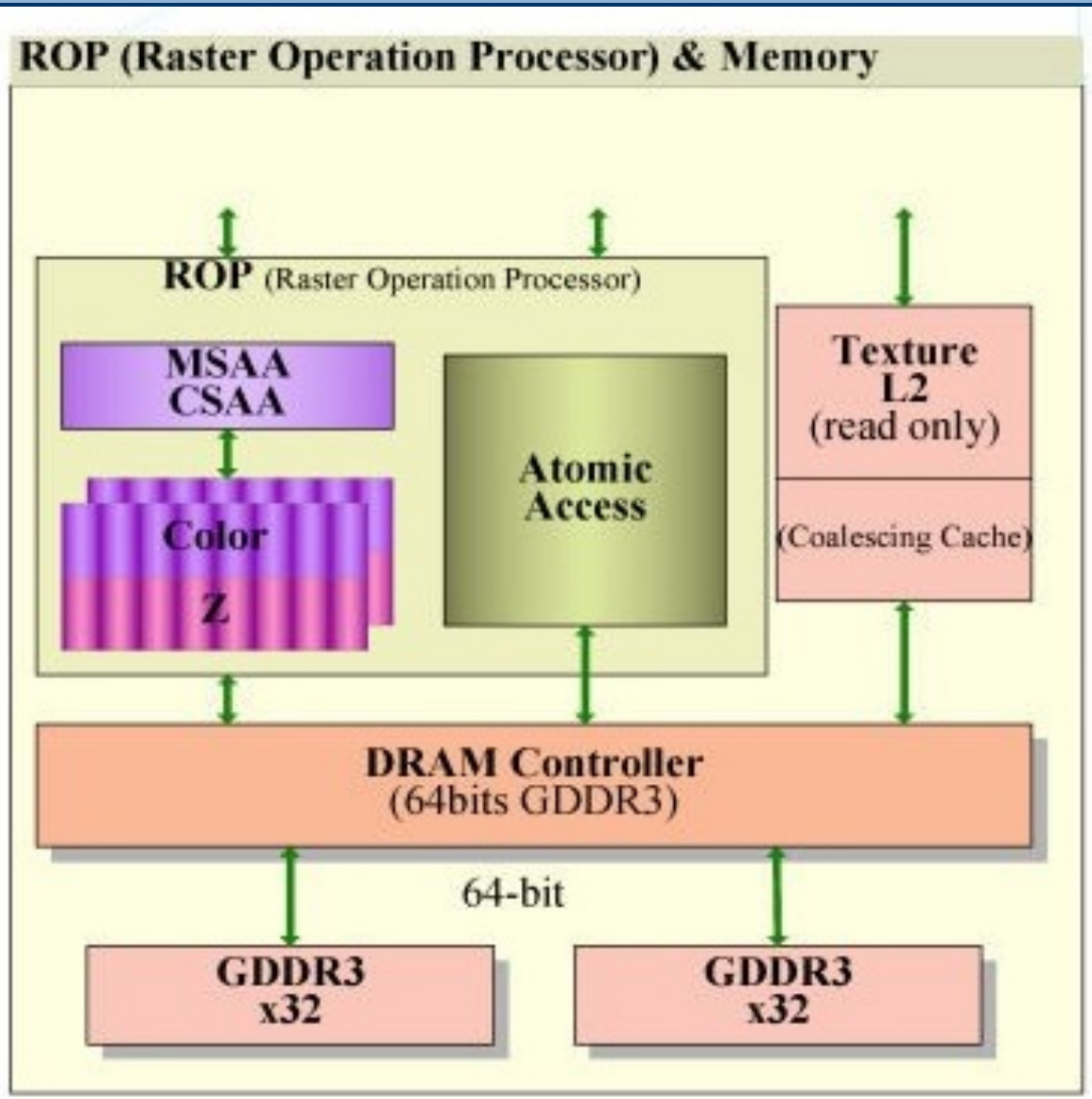


SM


Streaming Multiprocessor (SM) (=Processor Cluster)
Texture L1 Cache 8KB? (read only)
Texture Units
Texture Address Generator
Texture Filter Unit
Streaming Multiprocessor (SM)
Instruction L1 Cache (read only)
L1 Fill
Multithreaded Instruction Fetch and Issue Unit (MT Issue)
(Out-of-Order Thread Dispatch, up to 512 Threads active)
16KB Shared Memory
(16banks, read/write)
(Not used explicitly for pixel shader programs)
Double Precision Unit
Super Function Unit (SFU)
Streaming Processor (SP) 0
Streaming Processor 1
Streaming Processor 2
Streaming Processor 3
Streaming Processor 4
Streaming Processor 5
Streaming Processor 6
Streaming Processor 7
Register File(RF) 0
Register File 1
Register File 2
Register File 3
Register File 4
Register File 5
Register File 6
Register File 7
Load Texture
Register File
64KB(SM total)
16384 x 32bits registers /SM
Constant L1 Cache 8KB? (read only)
Instruction & Constant L2 Cache (read only)
Store to Memory
Load from Memory
Load/Store Unit

ROP

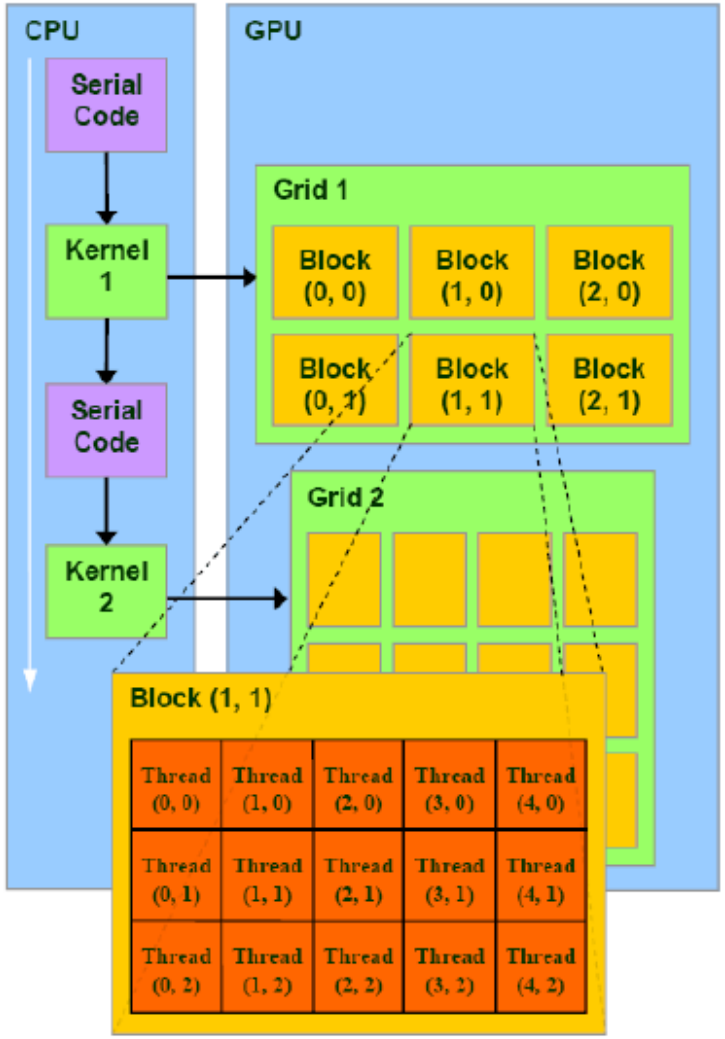
- 对DRAM进行访问
- TEXTURE机制
- 对global的atomic操作



CUDA 执行模型

- 将CPU作为主机(Host)，而GPU作为协处理器(Coprocessor) 或者设备（Device），从而让GPU来运行一些能够被高度线程化的程序。
- 在这个模型中，CPU与GPU协同工作，CPU负责进行逻辑性强的事务处理和串行计算，GPU则专注于执行高度线程化的并行处理任务。
- 一个完整的CUDA程序是由一系列的设备端kernel函数并行步骤和主机端的串行处理步骤共同组成的。

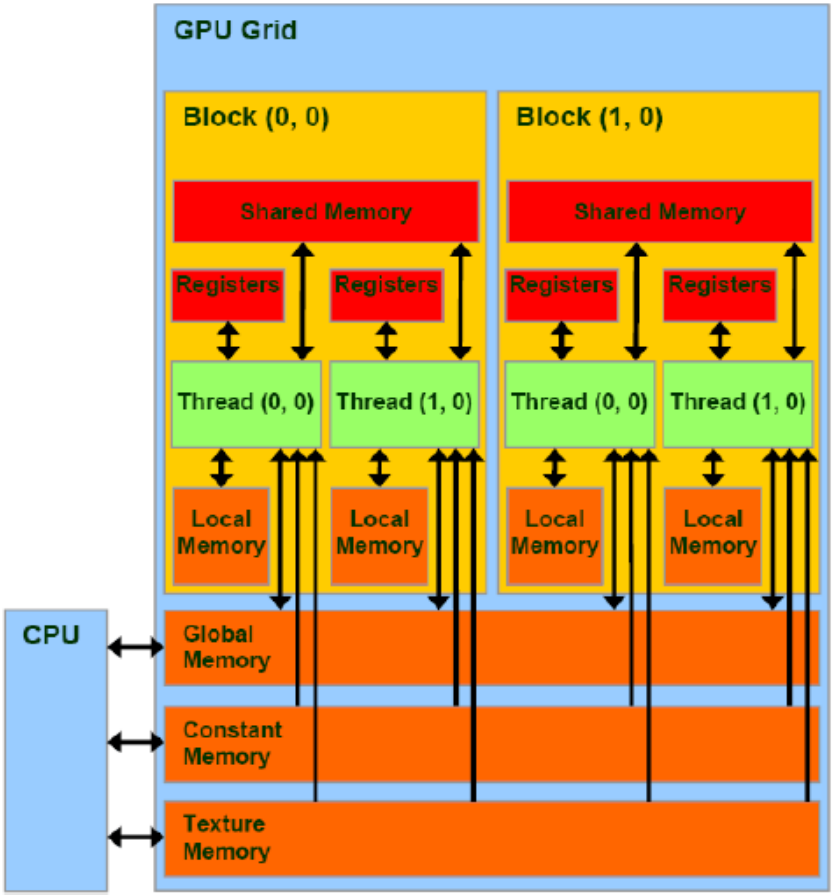
- grid运行在SPA上
- block运行在SM上
- thread运行在SP上



grid block thread

- **Kernel**不是一个完整的程序，而只是其中的一个关键并行计算步骤。
- **Kernel**以一个网格(Grid)的形式执行，每个网格由若干个线程块（block）组成，每一个线程块又由若干个线程(thread)组成。
- 一个**grid**最多可以有**65535 * 65535**个**block**
- 一个**block**总共最多可以有**512**个**thread**，在三个维度上的最大值分别为**512, 512**和**64**

- Register
- Local
- shared
- Global
- Constant
- Texture
- Host memory
- Pinned host memory



CUDA C语言

- 由Nvidia的CUDA编译器(nvcc)编译
- CUDA C不是C语言，而是对C语言进行扩展形成的变种
- 引入了函数类型限定符：__device__，__host__和__global__。
- 引入了变量限定符：__device__，__shared__和__constant__。
- 引入了内置矢量类型：char1，dim3，double2等
- 引入了内建变量：blockIdx，threadIdx，gridDim，blockDim和warpSize
- 引入了<<<>>>运算符
- 引入了一些函数：同步函数，原子函数，纹理函数等

主机端代码主要完成的功能：

- 启动**CUDA**
- 为输入数据分配内存空间
- 初始化输入数据
- 为**GPU**分配显存，存放输入数据
- 将内存输入数据拷贝到显存
- 为**GPU**分配显存，存放输出数据
- 调用**device**端的**kernel**计算
- 为**CPU**分配内存，存放输出数据
- 将显存结果读到内存
- 使用**CPU**进行其他处理
- 释放内存和显存空间
- 退出**CUDA**

设备端代码主要完成的功能

- 从显存读数据到**GPU**中
- 对数据处理
- 将处理后的数据写回显存

并行算法的负载均衡

❖ 负载不均衡代码示例

```
#pragma omp parallel for schedule(static)
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++) {
        float diff= pos[i]-pos[j];
            ...;//computing
        }
}
```

❖缓解负载不均衡代码示例

```
#pragma omp parallel for schedule(dynamic)
for(int i=0; i<n; i++){
    for(int j=0; j<n; j++) {
        float diff= pos[i]-pos[j];
            ...;//computing
        }
}
```